A REPORT

STOCK MARKET ANALYZER USING DATA STRUCTURES

Name of the Students	Registration No.
Mohammed Salman Tanveer	AP24110012087
M Bhargav	AP24110011912
B. Lokesh Manikanta	AP24110011917
B. Stevin Babu	AP24110011959

Prepared in the partial fulfillment of the

Project Based Learning of Course CSE 206 – CODING SKILLS II

Under The Guidance Of

DR. PRAKASH

SRM UNIVERSITY, AP

APRIL- 2026

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all those who have contributed to the successful completion of this project.

First and foremost, we are deeply grateful to our Honourable Vice Chancellor **Prof. Ch. Satish Kumar sir** and **Dean of the School of Engineering and Sciences Prof. C V Tomy sir** and the Head of the Department for Computer Science and Engineering **Dr.Murali Krishna Enduri** PhD Associate Professor providing us with the opportunity and necessary facilities to undertake this project. Their constant encouragement and commitment to academic excellence have been a great source of motivation throughout this work.

I would like to extend my heartfelt thanks to my faculty Mentor **Dr. Prakash** PhD for their valuable guidance, insightful suggestions, and continuous support during the course of the project. Their expertise and professional insights have played a crucial role in shaping the outcome of this work.

I also wish to convey my sincere appreciation to all my Faculty members of the department for their encouragement, constructive feedback, and invaluable academic guidance. Their continuous supervision and advice have been instrumental in the successful completion of this project.

Finally, I would like to thank all those who have directly or indirectly helped me during this project — my colleagues, friends, and family — for their constant support, understanding, and cooperation.

With Regards

MD Salman Tanveer
(AP24110012087)

M Bhargav
(AP24110011912)

B. Lokesh Manikanta
(AP24110011917)

B. Stevin Babu
(AP24110011959)

CERTIFICATE

This is to certify that the project entitled has been successfully completed by MD Salman Tanveer - (AP24110012087), M Bhargav-(AP24110011912),B. Lokesh Manikanta-(AP24110011917),B. Stevin Babu-(AP24110011959) as a required academic component of the course **CSE 206: CODING SKILLS II** during Semester 4 of Academic Year 2025-26 in the Department of Computer Science and Engineering at SRM University, AP. This work was executed under the guidance of the undersigned and is deemed to have successfully met all course requirements as of 30/04/2026.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING, SRM UNIVERSITY AP,

NEERUKONDA, MANGALAGIRI-522240,

ANDHRAPRADESH, INDIA.

(Signature of the Course Instructor)

DR.PRAKASH,

PROFESSOR,

Department of Computer Science and Engineering,

SRM University,AP.

ABSTRACT

Stock market analysis plays a crucial role in understanding price trends and making informed financial decisions. However, manually analyzing large amounts of stock data can be time-consuming and inefficient. To address this problem, this project presents a **Stock Market Analyzer** developed using multiple data structures and efficient algorithms in C++.

The system accepts stock prices as input and processes them using various data structures such as Linked Lists, Binary Search Trees (BST), Graphs, Stacks, Queues, Heaps, Sets, and Hash Maps. Each structure is used to perform a specific type of analysis, such as maintaining price history, sorting values, representing price transitions, and extracting meaningful insights.

The analyzer performs several operations including finding the Next Greater Element using a stack, computing Sliding Window Maximum using a deque, identifying Top K highest prices using a priority queue, calculating frequency using a hash map, and displaying unique sorted values using a set. A graph is also constructed to represent transitions between consecutive stock prices.

To ensure efficient processing and optimized performance, the system uses well-known algorithms with optimal time complexities. The output is displayed in a clear and structured format, making it easy to understand the behavior and trends of stock prices.

In conclusion, this project demonstrates how fundamental data structures and algorithms can be effectively applied to solve real-world problems in stock data analysis, improving efficiency, accuracy, and decision-making capabilities.

.

INTRODUCTION

In today's fast-paced financial world, analyzing stock market data is an essential task for understanding price trends and making informed investment decisions. A well-structured analysis helps in identifying patterns, predicting future movements, and extracting meaningful insights from large datasets. However, performing stock analysis manually is often time-consuming and inefficient, especially when dealing with continuously changing price values.

With the increasing volume of stock data, manual methods become impractical and prone to errors. It becomes difficult to track price history, identify trends, compare values, and perform multiple analyses simultaneously. Tasks such as finding maximum values in a given range, identifying next greater elements, or determining frequency distributions require efficient computational techniques. These challenges highlight the need for an automated and optimized solution.

This project introduces a **Stock Market Analyzer** that utilizes various data structures and algorithms to simplify and enhance the process of stock data analysis. The system is designed using structures such as Linked Lists, Binary Search Trees, Graphs, Stacks, Queues, Heaps, Sets, and Hash Maps. Each structure is used to perform specific operations efficiently, such as maintaining price history, sorting data, and analyzing relationships between stock prices.

The system accepts stock prices as input and processes them through different algorithms to generate meaningful outputs. It performs operations such as Next Greater Element detection, Sliding Window Maximum calculation, Top K price identification, frequency analysis, and unique sorted value extraction. Additionally, a graph is constructed to represent transitions between consecutive stock prices, helping visualize price movement patterns.

The results generated by the system are clear, structured, and easy to interpret. By combining multiple data structures and optimized algorithms, the project ensures efficient data processing and accurate analysis.

Overall, this project aims to reduce manual effort, improve computational efficiency, and provide a reliable solution for stock data analysis. It demonstrates how fundamental concepts of Data Structures and Algorithms can be effectively applied to solve real-world problems in financial analytics.

Main Aim of the Project

The main aim of this project is to develop a **Stock Market Analyzer** that can efficiently process and analyze stock price data using various data structures and algorithms. The system focuses on reducing manual effort, improving performance, and providing accurate insights by implementing optimized techniques such as stack-based analysis, sliding window methods, heap operations, and graph representations.

Objectives of the Project

- To design and develop an automated system for analyzing stock market price data.
- To minimize manual effort and reduce errors in stock data analysis.
- To efficiently store and manage stock prices using appropriate data structures.
- To analyze price trends and patterns using optimized algorithms.
- To identify key insights such as maximum values, frequency, and unique elements.
- To perform advanced operations like Next Greater Element and Sliding Window Maximum.
- To represent relationships between consecutive stock prices using graph structures.
- To improve performance and efficiency using data structure-based solutions.
- To present results in a clear, structured, and easy-to-understand format.
- To demonstrate the practical application of Data Structures and Algorithms in solving real-world financial problems.

Methodology

The Stock Market Analyzer is developed using a structured approach that integrates multiple data structures and efficient algorithms. The methodology focuses on analyzing stock price data effectively by following a step-by-step process.

1. Input Collection
The system begins by collecting stock price inputs from the user:
- Number of stock prices
- Individual price values

These inputs form the base dataset for analysis.

2. Initialization
Several data structures are initialized to store and process the data:
- Vector → Stores all stock prices
- Linked List → Maintains price history
- Binary Search Tree (BST) → Stores sorted data
- Graph → Represents price transitions

Each structure serves a specific purpose in analysis.

3. Data Insertion
All input prices are inserted into multiple data structures:
- Added to vector for general processing
- Inserted into linked list for history tracking
- Inserted into BST for sorted representation

This ensures data is available for different types of operations.

4. Graph Construction
A graph is built to represent relationships between consecutive prices:
- Each price is connected to the next price

- Helps in visualizing stock price movement

This provides insight into transitions and trends.

5. Algorithmic Analysis

The core processing is performed using efficient algorithms:

- Next Greater Element (Stack)
    - Finds the next higher value for each price
- Sliding Window Maximum (Deque)
    - Calculates maximum values within a fixed window
- Top K Elements (Heap / Priority Queue)
    - Identifies highest stock prices
- Frequency Analysis (Hash Map)
    - Counts occurrences of each price
- Unique Sorted Values (Set)
    - Extracts distinct sorted elements

6. Data Processing Optimization

The system ensures efficient performance by:

- Using optimal data structures for each task
- Reducing time complexity (mostly O(n))
- Avoiding redundant computations

This results in faster and more accurate analysis.

7. Output Display

Finally, the results are displayed in a structured format:

- List of stock prices
- Price history (Linked List)
- Sorted values (BST)
- Graph representation
- Analytical outputs:
    - Next Greater Elements
    - Sliding Window Maximum
    - Top K values
    - Frequency distribution
    - Unique sorted values

This methodology ensures that the system performs comprehensive stock analysis efficiently while demonstrating the effective use of multiple data structures and algorithms.

6. S/W AND H/W REQUIREMENTS

To implement and execute the Stock Market Analyzer, the following resources are required:

Software Requirements:

Component	Specification
Programming Language	C++ (C++11 or above)
Compiler	GCC / Visual Studio Code / Code::Blocks
Operating System	Windows / Linux / macOS
Required Libraries	iostream, vector, stack, deque, queue, set, unordered_map, algorithm

Hardware Requirements:

Component	Specification
Processor	Intel i3 or above
RAM	Minimum 4 GB
Storage	At least 100 MB free space
Input Device	Keyboard and Mouse
Output Device	Monitor

CODE

```
#include <bits/stdc++.h>
using namespace std;

/* ---------- LINKED LIST ---------- */
class HistoryList {
    struct Node {
        int price;
        Node* next;
        Node(int p) : price(p), next(nullptr) {}
    };
    Node* head;

public:
    HistoryList() { head = nullptr; }

    void add(int price) {
        Node* node = new Node(price);
        node->next = head;
        head = node;
    }

    void display() {
        Node* temp = head;
        cout << "History: ";
        while (temp) {
```

```
            cout << temp->price << " ";
            temp = temp->next;
        }
        cout << endl;
    }
};

/* ---------- BST ---------- */
class BST {
    struct Node {
        int val;
        Node* left;
        Node* right;
        Node(int v) : val(v), left(nullptr), right(nullptr) {}
    };
    Node* root;

    Node* insert(Node* node, int val) {
        if (!node) return new Node(val);
        if (val < node->val) node->left = insert(node->left, val);
        else node->right = insert(node->right, val);
        return node;
    }

    void inorder(Node* node) {
        if (!node) return;
        inorder(node->left);
        cout << node->val << " ";
        inorder(node->right);
    }

public:
    BST() { root = nullptr; }

    void insert(int val) { root = insert(root, val); }

    void display() {
        cout << "BST (sorted): ";
        inorder(root);
        cout << endl;
    }
};

/* ---------- GRAPH ---------- */
class Graph {
    unordered_map<int, vector<int>> adj;

public:
    void addEdge(int u, int v) {
        adj[u].push_back(v);
    }

    void display() {
        cout << "Graph (price transitions):\n";
        for (auto &p : adj) {
            cout << p.first << " -> ";
            for (int v : p.second) cout << v << " ";
            cout << endl;
```

```
        }
    }
};

/* ---------- ANALYZER ---------- */
class StockAnalyzer {
    vector<int> prices;

public:
    void addPrice(int p) {
        prices.push_back(p);
    }

    vector<int> getPrices() {
        return prices;
    }

    void showPrices() {
        cout << "Prices: ";
        for (int p : prices) cout << p << " ";
        cout << endl;
    }

    void nextGreater() {
        stack<int> st;
        vector<int> res(prices.size(), -1);

        for (int i = prices.size() - 1; i >= 0; i--) {
            while (!st.empty() && st.top() <= prices[i])
                st.pop();
            if (!st.empty()) res[i] = st.top();
            st.push(prices[i]);
        }

        cout << "Next Greater Elements: ";
        for (int x : res) cout << x << " ";
        cout << endl;
    }

    void slidingWindowMax(int k) {
        deque<int> dq;

        cout << "Sliding Window Max: ";
        for (int i = 0; i < prices.size(); i++) {

            while (!dq.empty() && dq.front() <= i - k)
                dq.pop_front();

            while (!dq.empty() && prices[dq.back()] < prices[i])
                dq.pop_back();

            dq.push_back(i);

            if (i >= k - 1)
                cout << prices[dq.front()] << " ";
        }
        cout << endl;
    }
```

```
    void topK(int k) {
        priority_queue<int> pq(prices.begin(), prices.end());

        cout << "Top " << k << " prices: ";
        while (k-- && !pq.empty()) {
            cout << pq.top() << " ";
            pq.pop();
        }
        cout << endl;
    }

    void frequency() {
        unordered_map<int, int> mp;
        for (int p : prices) mp[p]++;

        cout << "Frequency:\n";
        for (auto &p : mp)
            cout << p.first << " -> " << p.second << endl;
    }

    void uniqueSorted() {
        set<int> s(prices.begin(), prices.end());
        cout << "Unique Sorted: ";
        for (int x : s) cout << x << " ";
        cout << endl;
    }
};

/* ---------- MAIN ---------- */
int main() {

    StockAnalyzer sa;
    HistoryList history;
    BST tree;
    Graph graph;

    int n;
    cout << "Enter number of prices: ";
    cin >> n;

    vector<int> input(n);

    cout << "Enter prices:\n";
    for (int i = 0; i < n; i++) {
        cin >> input[i];
    }

    // Insert into structures
    for (int p : input) {
        sa.addPrice(p);
        history.add(p);
        tree.insert(p);
    }

    // Build graph
    for (int i = 0; i < n - 1; i++) {
        graph.addEdge(input[i], input[i + 1]);
```

```
    }

    // Display data
    sa.showPrices();
    history.display();
    tree.display();
    graph.display();

    sa.nextGreater();

    int k;
    cout << "Enter window size (k): ";
    cin >> k;
    sa.slidingWindowMax(k);

    int top;
    cout << "Enter top K: ";
    cin >> top;
    sa.topK(top);

    sa.frequency();
    sa.uniqueSorted();

    return 0;
}
```

Sample Output:

```
Enter number of prices: 10
Enter prices:
10 20 15 25 5 30 18 35 12 40
Prices: 10 20 15 25 5 30 18 35 12 40
History: 40 12 35 18 30 5 25 15 20 10
BST (sorted): 5 10 12 15 18 20 25 30 35 40
Graph (price transitions):
35 -> 12
30 -> 18
18 -> 35
5 -> 30
12 -> 40
25 -> 5
15 -> 25
20 -> 15
10 -> 20
Next Greater Elements: 20 25 25 30 30 35 35 40 40 -1
```

```
Next Greater Elements: 20 25 25 30 30 35 35 40 40 -1
Enter window size (k): 3
Sliding Window Max: 20 25 25 30 30 35 35 40
Enter top K: 4
Top 4 prices: 40 35 30 25
Frequency:
40 -> 1
35 -> 1
30 -> 1
18 -> 1
5 -> 1
12 -> 1
25 -> 1
15 -> 1
20 -> 1
10 -> 1
Unique Sorted: 5 10 12 15 18 20 25 30 35 40


=== Code Execution Successful ===
```

Conclusion

The Stock Market Analyzer successfully demonstrates how data structures and algorithmic techniques can be applied to solve real-world problems in an efficient and structured manner. By integrating multiple data structures such as Linked Lists, Binary Search Trees, Graphs, Stacks, Queues, Heaps, Sets, and Hash Maps, the system is able to perform comprehensive analysis of stock price data with minimal manual effort.

The project effectively handles tasks such as maintaining price history, sorting data, identifying trends, and extracting key insights like next greater elements, sliding window maximums, top K values, and frequency distribution. It also represents price transitions using graph structures, providing a better understanding of stock movement patterns.

The system reduces the complexity involved in manual analysis and ensures faster, more accurate results. It demonstrates how choosing the right data structure for a specific task can significantly improve performance and efficiency.

Although the current system works well for basic stock data analysis, it can be further enhanced by incorporating real-time stock data integration, graphical user interfaces, and predictive analytics using machine learning techniques.

Overall, this project highlights the practical application of Data Structures and Algorithms in financial data analysis. It proves that fundamental programming concepts, when applied effectively, can provide powerful solutions for analyzing and understanding complex datasets.